

Battery State of Charge Agentic AI Workshop

Participant Preparation and Hands On Guide

This guide prepares you for a one-hour MATLAB R2026a workshop using an AI coding agent and the experimental embedded-ai-deployment skill. The revised workflow begins with a read-only environment preflight, uses a prepared PyTorch reference when Python is unavailable, validates a MATLAB-native network in Simulink, and ends with host MEX verification and generated C-code inspection.

Workshop outcome

- Verify MATLAB, MCP connectivity, required products, and the embedded AI skill before beginning.
- Evaluate a prepared MATLAB-native LSTM State-of-Charge estimator.
- Import the PyTorch MLP as a MATLAB-native dlnetwork and validate it against original or prepared PyTorch reference outputs.
- Compare the baseline LSTM with the projected and fine-tuned LSTM checkpoint.
- Compare MATLAB, Simulink Normal Simulation, desktop MEX, and generated C-code evidence in gated phases.

Model path through the workshop

Part	Participant activity	Model used	Result
1	Evaluate the prepared MATLAB model	MATLAB-native LSTM	Baseline SOC accuracy
2	Import and verify a PyTorch model	PyTorch MLP rebuilt as dlnetwork	Framework equivalence
3	Validate system integration	MATLAB-native AI Simulink model	MATLAB versus Simulink evidence
4	Compare compression and generate code	Projected and fine-tuned LSTM	MEX and C-code evidence

Complete before the event

Complete this checklist before arriving. Installation during the workshop is a recovery option and may be blocked by company policy.

- ☐ Bring a Windows laptop and charger.
- ☐ Confirm internet access to GitHub, MathWorks, ChatGPT, and Microsoft Store resources.
- ☐ Confirm that you can sign in to MATLAB and to the ChatGPT desktop app or Codex CLI. A ChatGPT Free account can be used, subject to its available usage allowance.
- ☐ Obtain IT approval for local software, PowerShell, Git, and MATLAB Add-Ons.
- ☐ Install MATLAB R2026a and the required products below.
- ☐ Install the ChatGPT desktop app with Codex, or prepare the Codex CLI fallback.
- ☐ Download or clone the workshop repository. (You can also do this with the instructor during the workshop.)
- ☐ Open Aimbdworkshop.prj once before the event. (You can also do this with the instructor during the workshop.)

Required MATLAB products

Type	Required item	Used for
Release	MATLAB R2026a	Agent Skill workflow
Product	Simulink	System integration
Product	Deep Learning Toolbox	LSTM and imported network
Product	Statistics and Machine Learning Toolbox	Project workflow dependency
Product	MATLAB Coder	Host MEX and C code
Product	Simulink Coder	Simulink code generation
Product	Embedded Coder	Embedded C code and reports
Product	Fixed-Point Designer	Embedded numeric workflows
Add-On	Deep Learning Toolbox Model Compression Library	Network projection
Add-On	MATLAB Coder Interface for Deep Learning	Code generation from dlnetwork
Add-On	Deep Learning Toolbox Converter for PyTorch Models	Part 2 model import
Compiler	MATLAB-supported C++ compiler	Desktop MEX verification

Python and PyTorch

External Python and PyTorch are optional. The Deep Learning Toolbox Converter for PyTorch Models add-on is required for Part 2 import. If the MATLAB-managed PyTorch runtime is unavailable, the participant uses the prepared PyTorch reference inputs and outputs supplied with the workshop instead of installing Python packages during the session.

Install ChatGPT or Codex

Preferred Windows setup

Install the ChatGPT desktop app for Windows before the event. Use the Microsoft Store link or the following Windows Package Manager command if your organization permits it.

```
winget install --id 9PLM9XGG6VKS -s msstore
```

[Official ChatGPT desktop app for Windows](#)

ChatGPT account and Codex usage

A ChatGPT Free account can be used for the guided exercises, and a paid plan is not required to attend. Codex usage is not determined only by the words in a prompt; it also varies with model choice, task complexity, context length, reasoning, and tool calls. A long end-to-end task can therefore use more allowance than several short gated tasks.

Before the event, sign in and run one short Codex test. Check the usage indicator in Codex or use `/status` in Codex CLI when available. If a limit is reached, continue with the instructor-prepared scripts and results. Do not share account credentials. An OpenAI API key is not required for this workshop.

Exact token totals are not a workshop requirement and may not be shown for a ChatGPT-plan Codex session. The workshop therefore uses short phase prompts, approval gates, and prepared fallback evidence.

Codex CLI fallback

Use this fallback only when the desktop app is unavailable and your IT policy permits PowerShell downloads and user-local software installation.

```
powershell -ExecutionPolicy Bypass -c "irm https://chatgpt.com/codex/install.ps1 | iex"
```

The command uses the official OpenAI installer. ExecutionPolicy Bypass applies only to the PowerShell process; it does not override company security controls. If installation is blocked, pair with another participant or follow along with the instructor.

```
codex
```

At first launch, select Sign in with ChatGPT and complete authentication.

Download the lab files

Clone the repository before the event when possible.

```
git clone https://github.com/KantikaAscendas/Agentic-AI-Embedded-Skill-Workshop.git  
cd Agentic-AI-Embedded-Skill-Workshop
```

[Agentic AI Embedded Skill Workshop repository](#)

If Git is unavailable, use Code then Download ZIP on GitHub, extract the ZIP to a short writable path, and avoid working directly inside the ZIP file.

Install the MATLAB Agentic Toolkit

1. Open the MATLAB Agentic Toolkit repository and download agenticToolkitInstaller.mltbx.
2. Open the downloaded MLTBX file with MATLAB and approve the add-on installation.
3. In MATLAB, run the installer command below.

```
setupAgenticToolkit("install")
```

4. Select only the groups needed for this workshop: MATLAB Core, AI and Statistics, and Code Generation. Include the Simulink Agentic Toolkit because the workshop uses Simulink models.
5. For every new MATLAB session on Windows, initialize the Simulink Agentic Toolkit using the commands below.

```
addpath(fullfile(getenv("USERPROFILE"), ...  
    ".matlab", "agentic-toolkits", "simulink"))  
satk_initialize
```

The `satk_initialize` command adds the Simulink tools, validates the installation, and shares the current MATLAB session with the MCP server. You do not need to run `shareMATLABSession` separately after successful initialization.

Install the embedded AI skill

Open Codex in the workshop repository and send the following prompt. Review and approve the requested installation. Start a new Codex task after installation so the skill becomes available.

```
Install the embedded-ai-deployment skill from https://github.com/matlab/agent-skills-playground/tree/main/skills/embedded-ai-deployment
```

[Agent Skills Playground embedded AI deployment skill](#)

Verify the setup

- Open MATLAB R2026a and Aimbdworkshop.prj.
- Run the Windows `addpath` and `satk_initialize` commands shown above once in the current MATLAB session.
- Run `agentic_ai/scripts/workshopPreflight.m` and confirm that the final line is `READY=true`.
- If `READY=false`, keep the missing item visible and ask the instructor before continuing.
- Open Codex in the workshop repository and send the read-only preflight prompt below.

```
Use embedded-ai-deployment. Perform a read-only workshop preflight only. Do not inspect project data or models and do not modify files. Verify that the skill is available, a live MATLAB MCP session is connected, the exact release is MATLAB R2026a, required products and support packages are present, Simulink Agentic Toolkit tools are available, and agentic_ai/scripts/workshopPreflight.m passes. Report Python separately as managed runtime available, external Python available, or unavailable. Confirm that agentic_ai/generated and agentic_ai/results are writable. Return a concise PASS, WARNING, or FAIL table and finish with READY, READY WITH FALLBACK, or NOT READY. Stop and wait for approval.
```

Workshop workflow

Time	Activity	Participant evidence
0 to 8	Initialize MATLAB and run the read-only preflight	Readiness table
8 to 18	Part 1 baseline LSTM evaluation	SOC prediction and RMSE
18 to 32	Part 2 PyTorch MLP import and equivalence	Native dlnetwork and reference comparison
32 to 42	MATLAB versus Simulink comparison	Normal Simulation equivalence
42 to 55	Part 3 compression and host code verification	Size, RMSE delta, MEX result, C report
55 to 60	Review and questions	Evidence summary and next steps

Start prompt

Use embedded-ai-deployment. Work with this battery State-of-Charge repository in the connected MATLAB R2026a session. Preserve Exercise_1.m, Exercise_2.m, and Exercise_3.m. Create executable .m scripts under agentic_ai/generated and results under agentic_ai/results; also maintain a participant-facing Live Script summary when supported. Work one phase at a time, explain the evidence, and pause for approval. The participant workflow ends after host verification and C-code inspection. Begin with project discovery and a concise project summary only.

Part 1 prompt

Proceed with the prepared data split and baseline MATLAB-native LSTM evaluation. Do not retrain. Use the existing checkpoint and a representative subset of held-out test data. Report the test count and RMSE, save a prediction plot, then pause.

Part 2 prompt

Proceed with Part 2 using Part_2_AI_import/models/mlp_soc_model.pt2. Import it with the modern R2026a workflow and rebuild it as a MATLAB-native dlnetwork. Use the MATLAB-managed PyTorch runtime to generate original reference outputs when available. If Python or PyTorch is unavailable, do not install packages; use the prepared PyTorch reference inputs and outputs in the repository. Run the approved equivalence tests, report MAE, RMSE, and maximum absolute error, open the native network in Deep Network Designer, then pause. Do not treat this MLP as the final embedded model.

Why Simulink comparison

Simulink validates system-level integration before code generation: signal dimensions, single-precision datatypes, sample time, and output agreement. The required comparison uses the prepared MATLAB-native network. A masked direct-PyTorch Predict or S-function block is optional because R2026a size or datatype propagation can fail even when the imported network is numerically correct.

Simulink comparison prompt

Proceed with the prepared MATLAB-native AI Simulink model. Use the same single-precision inputs for MATLAB prediction and Simulink Normal Simulation. Report input size, datatype, sample time, RMSE, and maximum absolute error. If a direct PyTorch Simulink block fails size or datatype propagation, record an optional-path warning and continue with the MATLAB-native Simulink model without modifying the original exercise files. Save the comparison evidence, then pause.

Part 3 prompt

Proceed with Part 3. The primary goal is minimum flash and model size for a Cortex-M deployment pattern. Single-precision floating point is acceptable. Use the prepared projected and fine-tuned LSTM checkpoint rather than running the long compression sweep or full training. Compare it with the baseline on the same samples and report learnables, estimated parameter bytes, RMSE, and RMSE delta. Generate and validate a desktop MEX implementation before generating library-free C code and opening the code-generation report. Then pause.

Troubleshooting

Issue	Action
Company policy blocks installation	Do not bypass policy. Pair with a prepared participant or follow the instructor during the workshop.
Codex cannot see MATLAB	Run the Windows addpath command and satk_initialize in the MATLAB session you want Codex to use, then retry.
Skill is not available	Start a new Codex task after installation and explicitly say Use embedded-ai-deployment.
Preflight reports a missing item	Install the named MATLAB product or Add-On before continuing.
MEX compiler is missing	Install a MATLAB-supported compiler and configure mex -setup C++.
Build takes too long	Use the prepared checkpoint and instructor-generated artifacts; do not retrain.
OneDrive or antivirus slows builds	Use a short local writable project path for the workshop copy.
Python or PyTorch is unavailable	Use the prepared Part 2 PyTorch reference data. Do not install Python packages during the workshop.
Direct PyTorch Simulink block fails	Record an optional-path warning and continue with the prepared MATLAB-native Simulink model.
Codex usage limit is reached	Continue with the generated scripts, prepared results, and instructor demonstration. Do not share credentials.

Reference links

- [Workshop repository](#)
- [MATLAB Agentic Toolkit](#)
- [Agent Skills Playground](#)
- [Codex CLI guide](#)
- [ChatGPT desktop app for Windows](#)
- [Official Codex pricing and usage guidance](#)